Lab 9 : Digital CPU Foundations (Gates → Subcircuits → PC & Memory)

Objectives: By the end of this lab you will be able to:

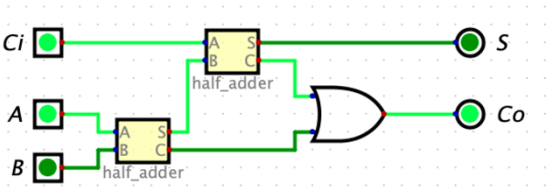
- Build a half adder and a full adder, then reuse them as subcircuits.
- Build and test a 4-bit ripple-carry adder.
- Add a latched zero flag to make a mini-ALU.
- Configure and test an 8×4 register file (2-read / 1-write).
- Build a 6-bit program counter, a 64×14 program ROM, and a 64×4 data RAM.
- Use a 4-to-1 multiplexer on a 4-bit data bus.

Note: In Digital during simulation: blue = 0, green = 1. Multi-bit wires show a decimal number. Stop the simulation before you edit a circuit.

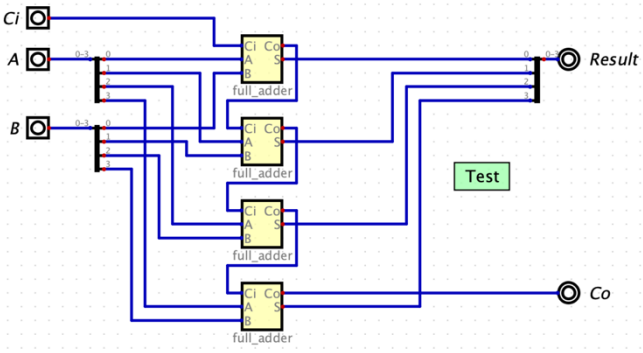
Lab 9.1 : Install and launch

1. If Digital is not installed and run Digital in Lab2. Then,
   Download the latest **Digital.jar** file from the official GitHub repository: https://github.com/hneemann/Digital/releases
   Download zip and unzip.
2. Ensure you have Java installed on your system (Mac, Windows, or Linux).
3. Run program run command in terminal
   ```
   java -jar Digital.jar
   ```
4. Keep every .dig file for this lab in one folder.

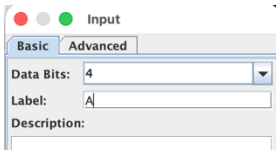
Lab 9.2 : Hierarchical Design: Adder / ALU



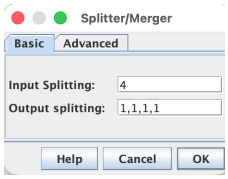
1. Make new circuit for half adder as: sum = A ⊕ B (A XOR B) , Carry = A · B (A AND B).
2. Save as "**halfAdder.dig".**
3. Create new empty file and save as "**fullAdder.dig".**
4. Make full adder from two half adder:
   4.1. Added to the new circuit via the *Components→Custom* menu→half_adder.
   4.2. Check the resut by "play"
   4.3.
5. **4-bit adder.** Create ripple-carry adder by make "**rcAdder.dig**" from full adder



   5.1. Add four full_adder subcircuits (Compoments -> Custom -> full_adder).
   5.2. Add Input, label **A** and **B**, set **Data Bits** to **4** (right click on input component)**.**



   5.3. Add two Splitter/Merger from Components -> Wires
   5.4. Set Splitter/Merger as Input Splitting:4 and Output splitting 1, 1, 1, 1



5.5. Connect as Diagram
5.6. Add Output label **Result,** set **Data Bits** to **4.**
5.7. Add Splitter/Merger for output and set to Input 1, 1, 1, 1 Output 4.
5.8. And Output for **Co**
5.9. Click “Play” and check result.
5.10. Save the circuit.

Check you circuit by check with table:

#	A	B	Ci	S	Co	Why
1	0	0	0	0	0	Both zero
2	1	1	0	2	0	Simple add
3	15	1	0	0	1	15+1=16 → wrap, carry
4	5	11	0	0	1	Lab zero test (5 + −5)
5	7	0	1	8	0	Carry-in only
6	15	15	1	15	1	15+15+1=31 → S=15, Co=1

Lab 9.3 : Building the Final 4-bit ALU

1. Our ALU is simple: it adds two numbers and checks if the result is zero.
2. Create a new circuit save file as “**miniALU.dig”.** Place your 4-bit adder sub-circuit (**rcAdder.dig)**.
3. Connect two 4-bit inputs, A and B, to the adder. The output is 4-bit label **Result**.
4. Add Compoments -> Wires -> Constantant Value. Set to **0** and connect to **Ci.**
5. **Zero Flag Logic**: We need to check if all 4 bits of the ALU_Result are zero.
   5.1. Use a splitter to separate the 4-bit ALU_Result (Input: 4, Output: 1, 1, 1, 1).
   5.2. Feed all 4 bits into a 4-input **NOR** gate (change Inumbers of inputs in properties to 4).
   5.3. The output of this **NOR** gate is your Zero_Flag. It will be 1 only when all input bits are 0.
6. **Flag Latching:**
   6.1. Add a Register component from Components -> Memory -> Register.
   6.2. In the Register's properties, set Data Bits to **1**.
   6.3. Connect the output of the 4-input **NOR** gate to the **D** input of this register.
   6.4. Add a new 1-bit Input to your ALU circuit and label it **FlagWrite**. Connect it to the en (enable) pin of the register. The flag will only be updated when this is high.
   6.5. Add another new 1-bit Input and label it **Clock**. Connect it to the C (clock) pin of the register.
   6.6. Add a 1-bit Output component and label it **Zero_Flag**.
   6.7. Connect the Q output of the register to this **Zero_Flag** output.
7. Save the circuit.

Check you circuit by:

1. Set for Zero: Input values that result in zero (e.g., A=5, B=11 for 5 + (-5) in 4-bit two's complement). Show that the combinational logic (the NOR gate output) is 1.
2. Latch the '1': With FlagWrite set to 1, advance the Clock by one tick. Show that the final Zero_Flag output is now latched and holding the value 1. (Drak green = 0, Light green = 1)
3. Hold the '1': Change the inputs to a non-zero result (e.g., A=2, B=3). Show that the NOR gate output is now 0.
4. Demonstrate Unlatch Condition: With FlagWrite set to 0, advance the Clock. Show that the Zero_Flag output *remains* 1, proving that the register is correctly holding the previous value and ignoring the new input.
5. Latch the '0': Set FlagWrite back to 1 and advance the Clock. Show that the Zero_Flag output now updates to 0.

Question 1: Why does the zero flag need to be latched?

Lab 9.4 : Register File

1. Create the Register File Circuit:

1.1. Create a new file and save it as "**registerFile.dig"**.
1.2. Add a Register File component from Components -> Memory - RAM -> Register File.
1.3. Configure Properties: Right-click the component and set:
    1.3.1. Data Bits: 4
    1.3.2. Address Bits: 3 (3 bits can address 8 unique registers, R0 to R7).
1.4. Add and connect input: : 4-bit to **Din** (data in), 1-bit to **we** (write enable), 3-bit **Rw** (register to be written), 1-bit to **C** (clock), 3-bit to **Ra** and **Rb**.
1.5. Add and connect output: 4-bit to Da and 4-bit to Db
1.6. Click "Play" for simulation. Right click to see the data.

Check you circuit by:

1. **Write to R2:**
    1.1. Set we (Write Enable) to **1**.
    1.2. Set Rw (Write Address) to **10** (binary for 2).
    1.3. Set Din (Data In) to **0111** (binary for 7).
    1.4. Advance the Clock one tick. The value 7 is now stored in R2.
    1.5. Right click on register file to see data.
    1.6. **Disable Write:** Set we back to **0**.
2. **Read from R2 on Port A:** Set Ra (Read Address A) to **10**. Show that the Da output is now **0111**.
3. **Read from R2 on Port B:** Set Rb (Read Address B) to **10**. Show that the Db output is also **0111**.
4. **Verify Independence:** Write a different value (e.g., 5) to a different register (e.g., R1, address 01) and show you can read it back without affecting the value in R2.

Question 2.1 : What is the purpose of register file?
Question 2.2: What are Din, WE, Rw, CLK, Ra, Rb, Da and Db?
Question 2.3: Why are Ra, Rb are 3 bits, but Da and Db 4 bits?

Lab 9.5: Program Counter and Memory (ROM and RAM)
Part A: Program Counter (PC)

1. New circuiot. Save as "**programCounter.dig".**
2. Components -> Memory -> Counter with preset. Data Bits = 6.
3. 6-bit Output labelled out. This address goes to the program ROM.
4. Connect en to Supply voltage (Components → Wires → Supply voltage) so the counter is always enabled.
5. Connect dir to Ground (Components → Wires → Ground) so it counts up.
6. Inputs: 1-bit C, 6-bit in (preset address), 1-bit ld, 1-bit clr.
7. Save the circuit.

Check you circuit:

1. Output count form 0 to 0x3F and back to 0 on every **clock** tick.
2. Set data **in** and **ld** = 1 and clock tick, output the data in
3. Clear to 0 when clr is 1 and clock tick

Question 3.1: Why have to connect PC's en pin to supply voltage?
Question 3.2: Why have to connect PC's dir pin to ground?
Question 3.3: What *ld* pin for?
Question 3.4: Why PC has 6-bit output not 4-bit as register file?

Part B: Program ROM

1. New circuit. Save as, "**programROM.dig**".
2. Components → Memory → ROM. Address Bits = 6. Data Bits = 14.
3. 6-bit Input to pin A (address). 14-bit Output from pin D (instruction).
4. Connect sel to Supply voltage so the ROM is always selected.

5. Right-click the ROM → Edit Content. Enter 14-bit instruction words. Unused addresses may stay 0.
6. Save the circuit.

Check: Load instruction into ROM, set address to see the correct output. Example:

Address (6-bit)	Instruction (14-bit, example)
0	0x0001
1	0x0002
2	0x3FFF

Question 3.5: Why data of ROM is 14 bits not 6 bits or 4 bits?
Question 3.6: Between a 14-bit instruction and a 12-bit instruction, which is better for this lab, and why?

Part C: Data RAM

1. New circuit. Save as dataRAM.dig.
2. Components → Memory → RAM → RAM, separated Ports. Address Bits = 6. Data Bits = 4.
3. Inputs: 6-bit A (address), 4-bit Din, 1-bit str (MemWrite), 1-bit ld (MemRead), 1-bit C (clock).
4. Output: 4-bit D (data out).
5. Play. Right-click the RAM to inspect stored values.

Use Digital pin names str and ld. MemWrite means str. MemRead means ld.

1. Store: set A and Din, str=1, one clock tick. Then str=0.
2. Load: set A, ld=1. D shows the stored value. A load with this component follows the ld pin; watch D after you set ld.

Question 3.7: Why can data RAM use 4-bit data while program ROM uses 14-bit data?

Lab 9.6: Multiplexer

A Multiplexer selects exactly one of N data inputs and forwards it to the output based on a selector (sel).

1. Create new file, "**Mux.dig**". Add multiplexer on canvas (Compoments -> Plexers -> Multiplexer).
2. Set data bits = 4, Selector bits = 2. This will create multiplexer with 4 4-bit input (in_0, in_1, in_2, in_3) and 2-bit selector 00 output from **in_0**, 01 output from **in_1**, 10 output from **n_2** and 11 output from n**_4**.
3. Add 4-bit input to **in_0**, **in_1**, **in_2**, **in_3**.
4. Add 4-bit output to **out**.
5. Set difference data for each input, set selector and observ the result.

LAB Submission

Submit these files, in one zip:

- halfAdder.dig
- fullAdder.dig
- rcAdder.dig
- miniALU.dig
- registerFile.dig
- programCounter.dig
- programROM.dig
- dataRAM.dig
- Mux.dig

Screen shot of each circuit.
Answers to Questions 1–3.7 can be typed in the form or written on this sheet.
Submission Form: https://forms.gle/R3rCSGqkRC5egCbFA